



MAPÚA MALAYAN COLLEGES MINDANAO

**SINGLE-CYCLE RISC-V SAP (SIMPLE-AS-POSSIBLE) COMPUTER DESIGN
AND DEVELOPMENT**

A Modular Assessment submitted
in Partial Fulfillment of the Requirements for the Course
CPE131L-1 Computer Architecture and Organization (Laboratory)

by:

Ghymes Louise C. Deocampo

Lyka Sophia S. Jardeleza

George Miguel V. Batalan

Tristan Shawn Alforque

Hym M. Verdan

Bachelor of Science in Computer Engineering

Course Professor

Marlon V. Maddara

College of Engineering and Architecture

February 2026

TABLE OF CONTENTS

TITLE PAGE	1
TABLE OF CONTENTS	2
INTRODUCTION	3
COMPLETE SOURCE CODE	4
PROGRAM OUTPUT	21
FINDINGS, OBSERVATIONS, AND COMMENTS	28
COMPLETE PROGRAM GUIDE	32

OVERVIEW OF THE APPLICATION

The Simple-As-Possible (SAP) Computer developed in this project is a single-cycle processor based on the RISC-V Instruction Set Architecture (ISA). The primary objective of this system is to demonstrate how a fundamental computer architecture executes machine-level instructions by integrating essential hardware components into a unified datapath. The processor consists of a Program Counter (PC), Instruction Memory, Register File, Arithmetic Logic Unit (ALU), Data Memory, Immediate Generator, Control Unit, ALU Control Unit, multiplexers, and necessary interconnecting buses. All operations are completed within a single clock cycle, ensuring that every instruction progresses through the entire datapath in one synchronous operation.

The processor implements the base RV32I subset of the RISC-V ISA and supports five primary instruction formats: R-Type, I-Type, S-Type, B-Type, and J-Type. R-Type instructions perform register-to-register arithmetic and logical operations such as addition, subtraction, and bitwise manipulation. I-Type instructions allow immediate arithmetic operations and load instructions. S-Type instructions are used for memory store operations. B-Type instructions support conditional branching, enabling the processor to alter program flow based on comparison results. Additionally, J-Type instructions (specifically the JAL instruction) enable unconditional jumps to target addresses while optionally storing the return address in a register, allowing support for function calls and control transfer operations. Including J-Type instructions enhances the processor's ability to manage program flow beyond simple conditional branching.

The architecture follows the single-cycle execution model, where each instruction completes all five fundamental stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB) within one clock cycle. Because every instruction completes in one cycle, the clock period must be long enough to accommodate the instruction with the longest critical path, typically load instructions. Although this limits maximum clock frequency compared to pipelined architectures, it significantly simplifies control logic and datapath analysis. Patterson and Hennessy (2021) explain that single-cycle processors are widely used in academic environments because they clearly illustrate the interaction between datapath components and control signals during instruction execution.

RISC-V was selected as the foundation of this SAP computer because it is an open-standard and modular ISA maintained by RISC-V International. Its clean 32-bit fixed instruction encoding and consistent field positioning simplify hardware decoding and control signal generation. The RV32I base integer instruction set follows a load-store architecture, meaning arithmetic operations are performed only between registers, while memory access occurs exclusively through load and store instructions (RISC-V International, 2019). This separation simplifies datapath design and makes hardware implementation more structured and predictable.

From a hardware design perspective, instruction decoding is achieved by analyzing the opcode, funct3, and funct7 fields. The Control Unit generates control signals such as RegWrite, MemRead, MemWrite, Branch, Jump, ALUSrc, and MemtoReg based on the instruction type. The ALU Control refines the exact ALU operation for arithmetic and logical instructions. Multiplexers determine ALU operand sources and select the appropriate write-back data path. For B-Type

and J-Type instructions, additional adder logic computes the branch or jump target address by adding the sign-extended immediate value to the current PC. The inclusion of jump control logic ensures proper update of the Program Counter during unconditional control transfers.

Implementing this processor strengthens understanding of key computer architecture concepts including instruction encoding, control signal derivation, datapath timing constraints, combinational and sequential logic integration, and modular system construction. By verifying that the processor executes R-Type, I-Type, S-Type, B-Type, and J-Type instructions correctly in any valid sequence within a single clock cycle, the system demonstrates both structural integrity and functional correctness.

Overall, the SAP computer developed in this laboratory project serves as a practical realization of modern processor fundamentals. It bridges theoretical instruction set design and hardware-level implementation. Through systematic design, integration, and validation, the project reinforces how machine-level instructions are translated into coordinated hardware operations while maintaining a simplified and organized architecture suitable for instructional purposes.

COMPLETE SOURCE CODE

Provide your complete source code here (Verilog code per data path component & test bench simulation code). Utilize the following format:

1. TopLevel_TB

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity ImmGen is
    Port ( INSTR    : in  STD_LOGIC_VECTOR (31 downto 0);
          IMM_OUT   : out STD_LOGIC_VECTOR (31 downto 0));
end ImmGen;

architecture Behavioral of ImmGen is
    signal opcode : STD_LOGIC_VECTOR(6 downto 0);
begin
    opcode <= INSTR(6 downto 0);

    process(INSTR, opcode)
    begin
        case opcode is
            when "0010011" | "0000011" | "1100111" =>
                if INSTR(31) = '1' then
                    IMM_OUT <= x"FFFFFF" & INSTR(31 downto 20);
                else
```

```

        IMM_OUT <= x"00000" & INSTR(31 downto 20);
    end if;

    when "0100011" =>
        if INSTR(31) = '1' then
            IMM_OUT <= x"FFFFFF" & INSTR(31 downto 25) & INSTR(11 downto
7);
        else
            IMM_OUT <= x"00000" & INSTR(31 downto 25) & INSTR(11 downto
7);
        end if;

    when "1100011" =>
        if INSTR(31) = '1' then
            IMM_OUT <= x"FFFFFF" & INSTR(7) & INSTR(30 downto 25) &
INSTR(11 downto 8) & '0';
        else
            IMM_OUT <= x"00000" & INSTR(7) & INSTR(30 downto 25) &
INSTR(11 downto 8) & '0';
        end if;

    when "1101111" =>
        if INSTR(31) = '1' then
            IMM_OUT <= x"FFF" & INSTR(19 downto 12) & INSTR(20) &
INSTR(30 downto 21) & '0';
        else
            IMM_OUT <= x"000" & INSTR(19 downto 12) & INSTR(20) &
INSTR(30 downto 21) & '0';
        end if;

    when others =>
        IMM_OUT <= (others => '0');
    end case;
end process;
end Behavioral;

```

2. PC (Program Counter)

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity PC is

```

```

    Port ( CLK      : in  STD_LOGIC;
          RST       : in  STD_LOGIC;
          NEXT_PC   : in  STD_LOGIC_VECTOR (31 downto 0);
          PC_OUT    : out STD_LOGIC_VECTOR (31 downto 0));
end PC;

architecture Behavioral of PC is
begin
    process(CLK, RST)
    begin
        if RST = '1' then
            PC_OUT <= (others => '0');
        elsif rising_edge(CLK) then
            PC_OUT <= NEXT_PC;
        end if;
    end process;
end Behavioral;

```

3. InstMem (Instruction Memory)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity InstMem is
    Port ( ADDRESS : in  STD_LOGIC_VECTOR (31 downto 0);
          INSTR    : out STD_LOGIC_VECTOR (31 downto 0));
end InstMem;

architecture Behavioral of InstMem is
    type ram_type is array (0 to 63) of STD_LOGIC_VECTOR(31 downto 0);

    signal RAM : ram_type := (
        0 => x"FFF00093",
        1 => x"00100113",
        2 => x"0020C463",
        3 => x"06300193",
        4 => x"0020E463",
        5 => x"06300213",
        6 => x"02000293",

```

```

        7 => x"0012A023",
        8 => x"0AA00313",
        9 => x"00628023",
        10 => x"00028383",
        11 => x"0002C403",
        12 => x"0002A483",
        13 => x"03D00513",
        14 => x"000505E7",
        15 => x"07700613",
        16 => x"00000013",
        others => x"00000000"
    );

begin

    process (ADDRESS)
        variable temp_addr : integer;
    begin
        temp_addr := to_integer(unsigned(ADDRESS(31 downto 2)));

        if temp_addr >= 0 and temp_addr <= 63 then
            INSTR <= RAM(temp_addr);
        else
            INSTR <= x"00000000";
        end if;
    end process;

end Behavioral;

```

4. RegisterFile (Register File)

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity RegisterFile is
    Port ( CLK          : in  STD_LOGIC;
          RST           : in  STD_LOGIC;
          REG_WRITE     : in  STD_LOGIC;

```

```

        RS1_ADDR    : in  STD_LOGIC_VECTOR (4 downto 0);
        RS2_ADDR    : in  STD_LOGIC_VECTOR (4 downto 0);
        RD_ADDR     : in  STD_LOGIC_VECTOR (4 downto 0);
        WRITE_DATA   : in  STD_LOGIC_VECTOR (31 downto 0);
        READ_DATA1   : out STD_LOGIC_VECTOR (31 downto 0);
        READ_DATA2   : out STD_LOGIC_VECTOR (31 downto 0)
    );
end RegisterFile;

architecture Behavioral of RegisterFile is

    type reg_array is array (0 to 31) of STD_LOGIC_VECTOR(31 downto 0);

    signal registers : reg_array := (others => x"00000000");

begin

    process(CLK)
    begin
        if rising_edge(CLK) then
            if RST = '1' then
                registers <= (others => x"00000000");
            else
                if (REG_WRITE = '1' and RD_ADDR /= "00000") then
                    registers(to_integer(unsigned(RD_ADDR))) <= WRITE_DATA;
                end if;
            end if;
        end if;
    end process;

    process(RS1_ADDR, RS2_ADDR, registers)
    begin
        if (RS1_ADDR = "00000") then
            READ_DATA1 <= x"00000000";
        else
            READ_DATA1 <= registers(to_integer(unsigned(RS1_ADDR)));
        end if;

        if (RS2_ADDR = "00000") then
            READ_DATA2 <= x"00000000";
        end if;
    end process;
end architecture Behavioral of RegisterFile;

```

```

        else

            READ_DATA2 <= registers(to_integer(unsigned(RS2_ADDR)));

        end if;

    end process;

end Behavioral;

```

5. ImmGen (Immediate Generator)

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity ImmGen is

    Port ( INSTR    : in  STD_LOGIC_VECTOR (31 downto 0);
          IMM_OUT   : out STD_LOGIC_VECTOR (31 downto 0));

end ImmGen;

architecture Behavioral of ImmGen is

    signal opcode : STD_LOGIC_VECTOR(6 downto 0);

begin

    opcode <= INSTR(6 downto 0);

    process(INSTR, opcode)

    begin

        case opcode is

            when "0010011" | "0000011" | "1100111" =>

                if INSTR(31) = '1' then

                    IMM_OUT <= x"FFFFFF" & INSTR(31 downto 20);

                else

                    IMM_OUT <= x"000000" & INSTR(31 downto 20);

                end if;

            when "0100011" =>

                if INSTR(31) = '1' then

                    IMM_OUT <= x"FFFFFF" & INSTR(31 downto 25) & INSTR(11 downto
7);

                else

                    IMM_OUT <= x"000000" & INSTR(31 downto 25) & INSTR(11 downto
7);

                end if;

        end case;

    end process;

end Behavioral;

```

```

        when "1100011" =>
            if INSTR(31) = '1' then
                IMM_OUT <= x"FFFFFF" & INSTR(7) & INSTR(30 downto 25) &
INSTR(11 downto 8) & '0';
            else
                IMM_OUT <= x"00000" & INSTR(7) & INSTR(30 downto 25) &
INSTR(11 downto 8) & '0';
            end if;

        when "1101111" =>
            if INSTR(31) = '1' then
                IMM_OUT <= x"FFF" & INSTR(19 downto 12) & INSTR(20) &
INSTR(30 downto 21) & '0';
            else
                IMM_OUT <= x"000" & INSTR(19 downto 12) & INSTR(20) &
INSTR(30 downto 21) & '0';
            end if;

        when others =>
            IMM_OUT <= (others => '0');
        end case;
    end process;
end Behavioral;

```

6. Mux2to1 (2 to 1 Multiplexer)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Mux2to1 is
    Port ( SEL : in  STD_LOGIC;
          A   : in  STD_LOGIC_VECTOR (31 downto 0);
          B   : in  STD_LOGIC_VECTOR (31 downto 0);
          X   : out STD_LOGIC_VECTOR (31 downto 0)
        );
end Mux2to1;

architecture Behavioral of Mux2to1 is
begin
    X <= A when (SEL = '0') else B;
end Behavioral;

```

7. ALUControl (Arithmetic Logic Unit Control)

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity ALUControl is
    Port ( ALU_OP      : in  STD_LOGIC_VECTOR (1 downto 0);
          FUNCT3      : in  STD_LOGIC_VECTOR (2 downto 0);
          FUNCT7      : in  STD_LOGIC_VECTOR (6 downto 0);
          ALU_CTRL    : out STD_LOGIC_VECTOR (3 downto 0));
end ALUControl;

architecture Behavioral of ALUControl is
begin
    process(ALU_OP, FUNCT3, FUNCT7)
    begin
        case ALU_OP is
            when "00" =>
                ALU_CTRL <= "0000";

            when "01" =>
                ALU_CTRL <= "0001";

            when "10" | "11" =>
                case FUNCT3 is
                    when "000" =>
                        if (ALU_OP = "10" and FUNCT7(5) = '1') then
                            ALU_CTRL <= "0001";
                        else
                            ALU_CTRL <= "0000";
                        end if;

                    when "001" =>
                        ALU_CTRL <= "0101";

                    when "010" =>
                        ALU_CTRL <= "1000";

                    when "011" =>

```

```

        ALU_CTRL <= "1001";

    when "100" =>
        ALU_CTRL <= "0100";

    when "101" =>
        if (FUNCT7(5) = '1') then
            ALU_CTRL <= "0111";
        else
            ALU_CTRL <= "0110";
        end if;

    when "110" =>
        ALU_CTRL <= "0011";

    when "111" =>
        ALU_CTRL <= "0010";

    when others =>
        ALU_CTRL <= "0000";
    end case;

    when others =>
        ALU_CTRL <= "0000";
    end case;
end process;
end Behavioral;

```

8. Adder32

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Adder32 is
    Port ( A : in  STD_LOGIC_VECTOR (31 downto 0);
          B : in  STD_LOGIC_VECTOR (31 downto 0);
          X : out STD_LOGIC_VECTOR (31 downto 0));
end Adder32;

```

```

architecture Behavioral of Adder32 is
begin
    X <= std_logic_vector(unsigned(A) + unsigned(B));
end Behavioral;

```

9. Mux3to1 (3 to 1 Multiplexer)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Mux3to1 is
    Port ( in0      : in  STD_LOGIC_VECTOR (31 downto 0);
          in1      : in  STD_LOGIC_VECTOR (31 downto 0);
          in2      : in  STD_LOGIC_VECTOR (31 downto 0);
          sel      : in  STD_LOGIC_VECTOR (1 downto 0);
          out_mux  : out STD_LOGIC_VECTOR (31 downto 0));
end Mux3to1;

```

```

architecture Behavioral of Mux3to1 is
begin
    process(in0, in1, in2, sel)
    begin
        if sel = "00" then
            out_mux <= in0;
        elsif sel = "01" then
            out_mux <= in1;
        elsif sel = "10" then
            out_mux <= in2;
        else
            out_mux <= (others => '0');
        end if;
    end process;
end Behavioral;

```

10. MainControl

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MainControl is
    Port ( OPCODE      : in  STD_LOGIC_VECTOR (6 downto 0);

```

```

    FUNCT3      : in  STD_LOGIC_VECTOR (2 downto 0);
    ZERO        : in  STD_LOGIC;
    LESS        : in  STD_LOGIC;
    LESS_U      : in  STD_LOGIC;
    PC_SRC      : out STD_LOGIC_VECTOR (1 downto 0);
    MEM_READ    : out STD_LOGIC;
    MEM_TO_REG  : out STD_LOGIC_VECTOR (1 downto 0);
    MEM_WRITE   : out STD_LOGIC;
    ALU_SRC     : out STD_LOGIC;
    REG_WRITE   : out STD_LOGIC;
    ALU_OP      : out STD_LOGIC_VECTOR (1 downto 0));

end MainControl;

```

architecture Behavioral of MainControl is

begin

```

    process(OPCODE, FUNCT3, ZERO, LESS, LESS_U)

```

```

    begin

```

```

        PC_SRC      <= "00";

```

```

        MEM_READ    <= '0';

```

```

        MEM_TO_REG  <= "00";

```

```

        MEM_WRITE   <= '0';

```

```

        ALU_SRC     <= '0';

```

```

        REG_WRITE   <= '0';

```

```

        ALU_OP      <= "00";

```

```

    case OPCODE is

```

```

        when "0110011" =>

```

```

            REG_WRITE <= '1';

```

```

            ALU_OP    <= "10";

```

```

        when "0010011" =>

```

```

            ALU_SRC    <= '1';

```

```

            REG_WRITE  <= '1';

```

```

            ALU_OP     <= "11";

```

```

        when "0000011" =>

```

```

            ALU_SRC    <= '1';

```

```

            MEM_TO_REG <= "01";

```

```

            REG_WRITE  <= '1';

```

```

MEM_READ    <= '1';

ALU_OP      <= "00";

when "0100011" =>

    ALU_SRC    <= '1';

    MEM_WRITE  <= '1';

    ALU_OP     <= "00";

when "1100011" =>

    ALU_OP <= "01";

    PC_SRC <= "00";

case FUNCT3 is
    when "000" =>
        if ZERO = '1' then PC_SRC <= "01"; end if;
    when "001" =>
        if ZERO = '0' then PC_SRC <= "01"; end if;
    when "100" =>
        if LESS = '1' then PC_SRC <= "01"; end if;
    when "101" =>
        if LESS = '0' then PC_SRC <= "01"; end if;
    when "110" =>
        if LESS_U = '1' then PC_SRC <= "01"; end if;
    when "111" =>
        if LESS_U = '0' then PC_SRC <= "01"; end if;
    when others =>
        PC_SRC <= "00";
end case;

when "1101111" =>

    REG_WRITE  <= '1';

    MEM_TO_REG <= "10";

    PC_SRC     <= "01";

when "1100111" =>

    ALU_SRC    <= '1';

    REG_WRITE  <= '1';

    MEM_TO_REG <= "10";

```

```

        ALU_OP      <= "00";

        PC_SRC      <= "10";

        when others =>

            end case;

        end process;

end Behavioral;

```

11. DataMem (Data Memory)

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity DataMem is

    Port ( CLK          : in  STD_LOGIC;

          RST           : in  STD_LOGIC;

          MEM_WRITE     : in  STD_LOGIC;

          MEM_READ      : in  STD_LOGIC;

          FUNCT3        : in  STD_LOGIC_VECTOR (2 downto 0);

          ADDRESS       : in  STD_LOGIC_VECTOR (31 downto 0);

          WRITE_DATA    : in  STD_LOGIC_VECTOR (31 downto 0);

          READ_DATA     : out STD_LOGIC_VECTOR (31 downto 0));

end DataMem;

architecture Behavioral of DataMem is

    type ram_type is array (0 to 63) of STD_LOGIC_VECTOR(31 downto 0);

    signal RAM : ram_type := (others => x"00000000");

begin

    process(CLK)

        variable temp_addr : integer;

    begin

        if rising_edge(CLK) then

            if RST = '1' then

                RAM <= (others => x"00000000");

            elsif MEM_WRITE = '1' then

                temp_addr := to_integer(unsigned(ADDRESS(31 downto 2)));

                if temp_addr >= 0 and temp_addr <= 63 then

```

```

case FUNCT3 is
    when "010" =>
        RAM(temp_addr) <= WRITE_DATA;

    when "001" =>
        if ADDRESS(1) = '0' then
            RAM(temp_addr)(15 downto 0) <= WRITE_DATA(15
downto 0);
        else
            RAM(temp_addr)(31 downto 16) <= WRITE_DATA(15
downto 0);
        end if;

    when "000" =>
        if ADDRESS(1 downto 0) = "00" then
            RAM(temp_addr)(7 downto 0) <= WRITE_DATA(7
downto 0);
        elsif ADDRESS(1 downto 0) = "01" then
            RAM(temp_addr)(15 downto 8) <= WRITE_DATA(7
downto 0);
        elsif ADDRESS(1 downto 0) = "10" then
            RAM(temp_addr)(23 downto 16) <= WRITE_DATA(7
downto 0);
        else
            RAM(temp_addr)(31 downto 24) <= WRITE_DATA(7
downto 0);
        end if;

    when others => null;
end case;
end if;
end if;
end if;
end process;

process(ADDRESS, RAM, MEM_READ, FUNCT3)
    variable temp_addr : integer;
    variable raw_word  : STD_LOGIC_VECTOR(31 downto 0);
begin
    if MEM_READ = '1' then
        temp_addr := to_integer(unsigned(ADDRESS(31 downto 2)));

```

```

    if temp_addr >= 0 and temp_addr <= 63 then
        raw_word := RAM(temp_addr);

        case FUNCT3 is
            when "010" =>
                READ_DATA <= raw_word;

            when "001" =>
                if ADDRESS(1) = '0' then
                    std_logic_vector(resize(signed(raw_word(15 downto 0)), 32));
                    READ_DATA <=
                else
                    std_logic_vector(resize(signed(raw_word(31 downto 16)), 32));
                    READ_DATA <=
                end if;

            when "101" =>
                if ADDRESS(1) = '0' then
                    READ_DATA <= x"0000" & raw_word(15 downto 0);
                else
                    READ_DATA <= x"0000" & raw_word(31 downto 16);
                end if;

            when "000" =>
                if ADDRESS(1 downto 0) = "00" then
                    std_logic_vector(resize(signed(raw_word(7 downto 0)), 32));
                    READ_DATA <=
                elsif ADDRESS(1 downto 0) = "01" then
                    std_logic_vector(resize(signed(raw_word(15 downto 8)), 32));
                    READ_DATA <=
                elsif ADDRESS(1 downto 0) = "10" then
                    std_logic_vector(resize(signed(raw_word(23 downto 16)), 32));
                    READ_DATA <=
                else
                    std_logic_vector(resize(signed(raw_word(31 downto 24)), 32));
                    READ_DATA <=
                end if;

            when "100" =>
                if ADDRESS(1 downto 0) = "00" then
                    READ_DATA <= x"000000" & raw_word(7 downto 0);
                elsif ADDRESS(1 downto 0) = "01" then

```

```

        READ_DATA <= x"000000" & raw_word(15 downto 8);
    elsif ADDRESS(1 downto 0) = "10" then
        READ_DATA <= x"000000" & raw_word(23 downto 16);
    else
        READ_DATA <= x"000000" & raw_word(31 downto 24);
    end if;

    when others =>
        READ_DATA <= (others => '0');
    end case;
else
    READ_DATA <= x"00000000";
end if;
else
    READ_DATA <= x"00000000";
end if;
end process;

end Behavioral;

```

12. ALU (Arithmetic Logic Unit)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ALU is
    Port ( OP_A      : in  STD_LOGIC_VECTOR (31 downto 0);
          OP_B      : in  STD_LOGIC_VECTOR (31 downto 0);
          ALU_CTRL   : in  STD_LOGIC_VECTOR (3 downto 0);
          ALU_RESULT : out STD_LOGIC_VECTOR (31 downto 0);
          ZERO       : out STD_LOGIC;
          LESS       : out STD_LOGIC;
          LESS_U     : out STD_LOGIC
        );
end ALU;

```

architecture Behavioral of ALU is

```

    signal result_temp : STD_LOGIC_VECTOR(31 downto 0);

```

```

begin

    process(OP_A, OP_B, ALU_CTRL)
        variable shift_amt : integer;
    begin
        result_temp <= (others => '0');

        case ALU_CTRL is
            when "0000" =>
                result_temp <= std_logic_vector(signed(OP_A) + signed(OP_B));

            when "0001" =>
                result_temp <= std_logic_vector(signed(OP_A) - signed(OP_B));

            when "0010" =>
                result_temp <= OP_A and OP_B;

            when "0011" =>
                result_temp <= OP_A or OP_B;

            when "0100" =>
                result_temp <= OP_A xor OP_B;

            when "0101" =>
                shift_amt := to_integer(unsigned(OP_B(4 downto 0)));
                result_temp <= std_logic_vector(shift_left(unsigned(OP_A),
shift_amt));

            when "0110" =>
                shift_amt := to_integer(unsigned(OP_B(4 downto 0)));
                result_temp <= std_logic_vector(shift_right(unsigned(OP_A),
shift_amt));

            when "0111" =>
                shift_amt := to_integer(unsigned(OP_B(4 downto 0)));
                result_temp <= std_logic_vector(shift_right(signed(OP_A),
shift_amt));

            when "1000" =>
                if signed(OP_A) < signed(OP_B) then

```

```

        result_temp <= x"00000001";
    else
        result_temp <= x"00000000";
    end if;

    when "1001" =>
        if unsigned(OP_A) < unsigned(OP_B) then
            result_temp <= x"00000001";
        else
            result_temp <= x"00000000";
        end if;

    when others =>
        result_temp <= (others => '0');
    end case;
end process;

ALU_RESULT <= result_temp;

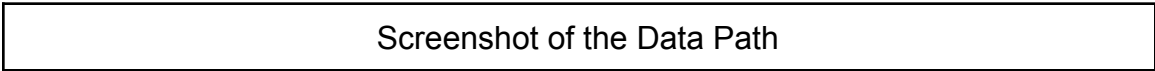
ZERO <= '1' when (result_temp = x"00000000") else '0';

LESS <= '1' when (signed(OP_A) < signed(OP_B)) else '0';
LESS_U <= '1' when (unsigned(OP_A) < unsigned(OP_B)) else '0';

end Behavioral;
```

PROGRAM OUTPUT

Provide here the output of your design by inserting screenshots of the data path, including screenshots of each component, and the output waveform of the simulation with clear and in-depth interpretation of results. Use as many pages as needed and provide clear descriptions for each provided image.



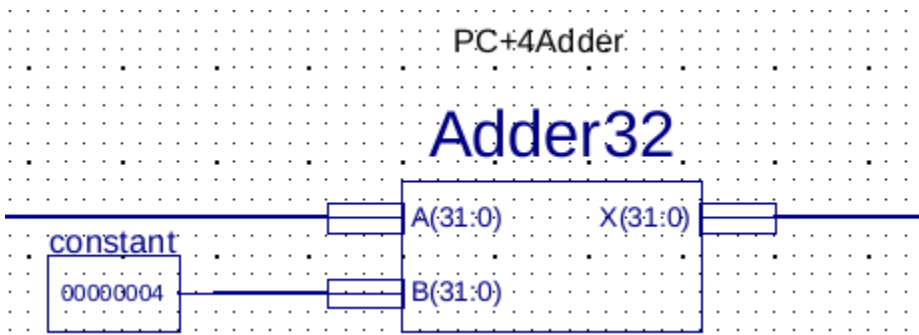


A block diagram of a PC (Program Counter) component. The block is labeled "PC" at the top. It has three inputs on the left: "CLK" (clock), "RST" (reset), and "NEXT_PC(31:0)" (next program counter value). It has one output on the right: "PC_OUT(31:0)" (current program counter value).

Located at the upper left.
Has:

- The Program Counter (PC) stores the address of the current instruction being executed. At every clock cycle, it sends this address to the instruction memory to fetch the next instruction. The PC is updated either with $PC + 4$ for sequential execution or with a branch or jump target address when control flow

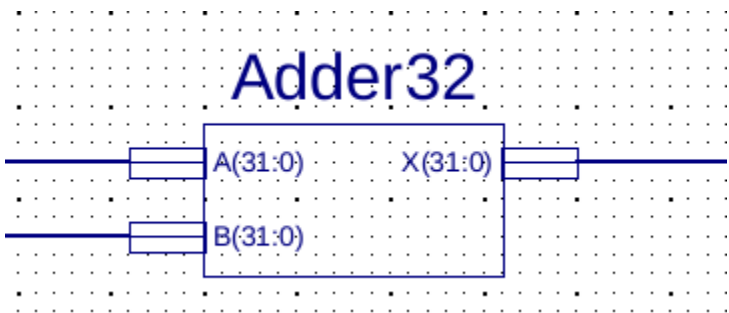
instructions occur.



PC + 4 Adder (Adder32)

- Labeled as Adder32 near the top.
- Adds a constant value (likely 4) to the current PC.
- Generates the sequential next instruction address.

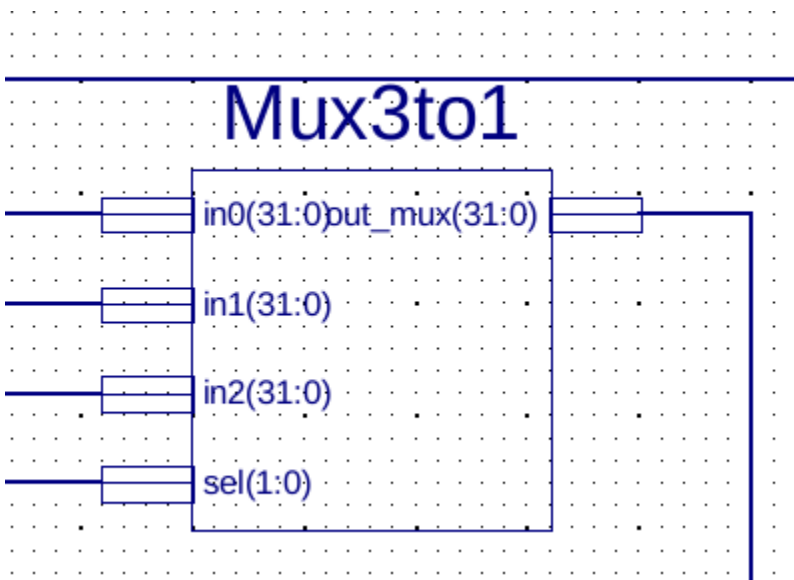
The PC + 4 adder calculates the address of the next sequential instruction by adding 4 bytes to the current PC value. Since RISC-V instructions are 32 bits long, each instruction occupies four bytes in memory. This adder ensures continuous instruction execution during normal program flow.



Branch/Jump Adder (Second Adder32)

- Another Adder32 below the PC+4 adder.
- Adds PC and immediate offset (for branch or jump target).
- Used for B-Type and J-Type address calculation.

A second 32-bit adder calculates the target address for branch and jump instructions. It adds the current PC value with the sign-extended immediate value generated from the instruction. This allows the processor to change program flow when a branch condition is met or when a jump instruction is executed.



Mux3to1 (PC Source Selector)

- Located near the top center.
- Selects the next PC value from:
- PC + 4
- Branch target
- Jump target
- Controlled by control signals.
- Determines final `NEXT_PC`.

The 3-to-1 multiplexer selects the next value for the Program Counter. The possible inputs include PC + 4, the branch target address, or the jump target address. The selection is controlled by signals generated from the main control unit.



Instruction Memory (InstMem)

- Located mid-left.

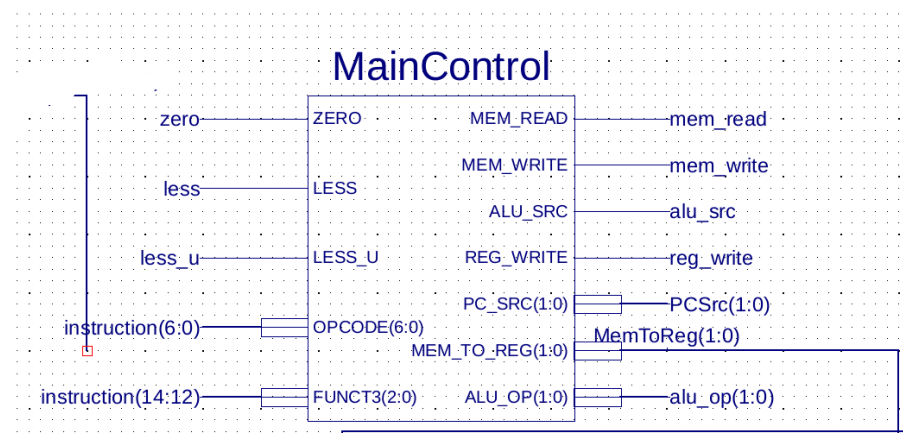
Input:

- Address from PC

Output:

- `instruction[31:0]`
- Provides the fetched instruction.

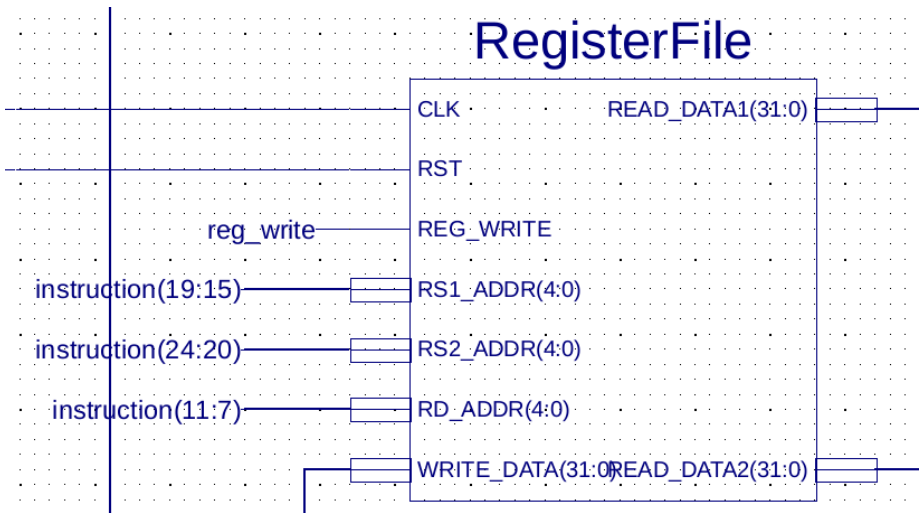
The Instruction Memory stores the program instructions. It receives the instruction address from the Program Counter and outputs the corresponding 32-bit instruction. This component represents the instruction fetch stage of the processor.



Main Control Unit (MainControl)

- Central component.
- Input:
 - `instruction[6:0]` (opcode)
- Outputs control signals such as:
 - `mem_read`
 - `mem_write`
 - `reg_write`
 - `mem_to_reg`
 - `alu_src`
 - `branch`
 - `jump`
 - `alu_op[1:0]`
- This unit determines instruction type.

The Main Control Unit interprets the instruction opcode and generates control signals that coordinate the entire datapath. These signals control operations such as register writing, memory access, ALU operations, and branching, ensuring the correct execution of each instruction type



Register File

- Located lower-left.

Inputs:

- `rs1`
- `rs2`
- `rd`
- `reg_write`

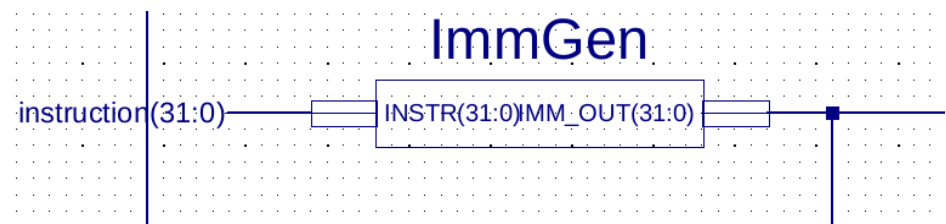
Outputs:

- `read_data1`

- read_data2

Stores 32 registers (x0–x31).

The Register File stores the processor's general-purpose registers. It allows two registers to be read simultaneously and one register to be written in a single cycle. The registers store operands for ALU operations and results produced during instruction execution.



Immediate Generator (ImmGen)

Below Register File.

Input:

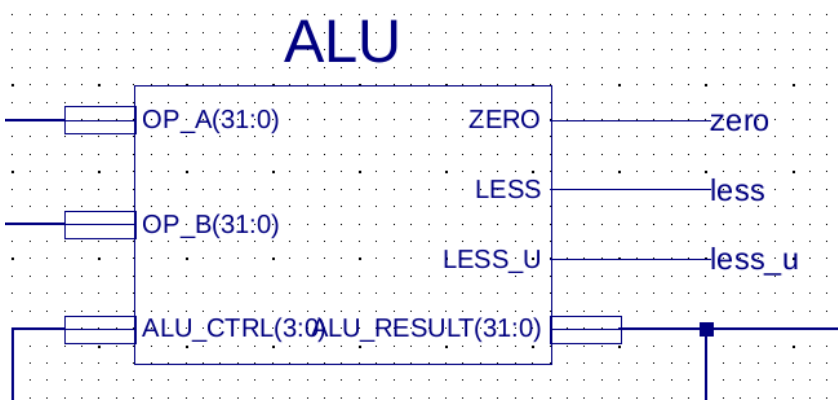
- Full instruction

Output:

- Sign-extended immediate value

Supports I, S, B, and J formats.

The Immediate Generator extracts and sign-extends immediate values from instructions. These values are used for operations such as address calculations, immediate arithmetic operations, and branch offsets.



Arithmetic Logical Unit (ALU)

Located upper-right.

Inputs:

- Operand A (register)
- Operand B (register or immediate)

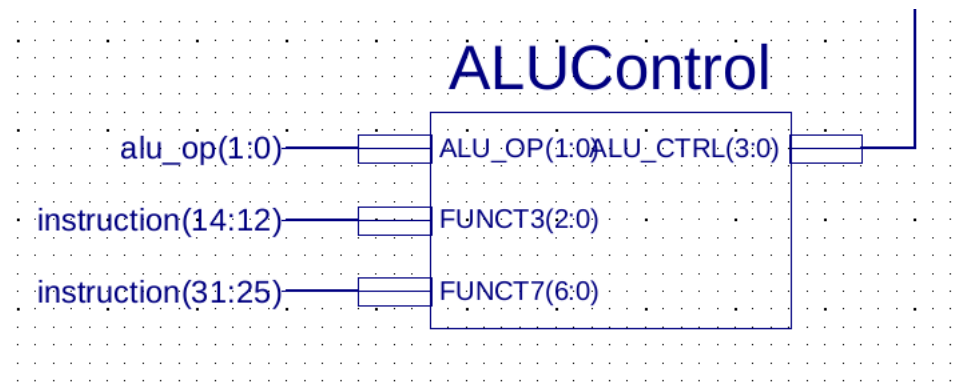
- ALU control signal

Outputs:

- ALU result
- zero
- less
- less_u

Performs arithmetic and logic operations.

The ALU performs arithmetic and logical operations such as addition, subtraction, and comparisons. It processes input operands from registers or immediate values and produces results used for register updates, memory addressing, or branch decisions.



ALU Control

Right of MainControl.

Inputs:

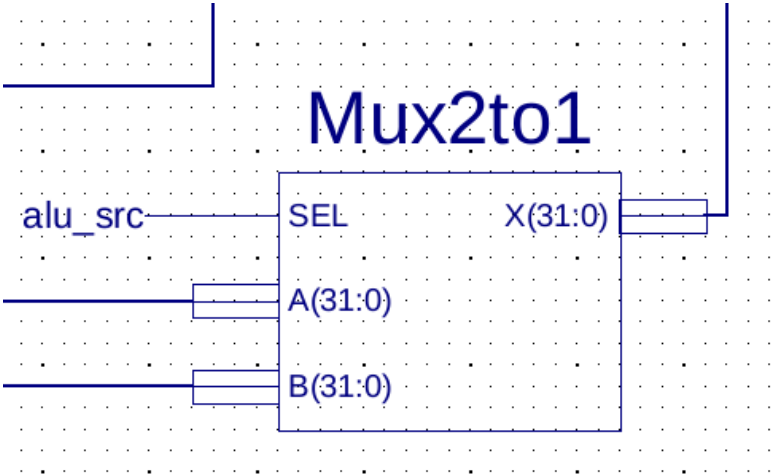
- `alu_op`
- `funct3`
- `funct7`

Output:

- `alu_ctrl`

Determines exact ALU operation (ADD, SUB, AND, OR, etc.)

The ALU Control Unit determines the exact operation the ALU should perform. It receives signals from the Main Control Unit along with function fields from the instruction and generates the appropriate ALU operation code.



Mux2to1 (ALU Source)

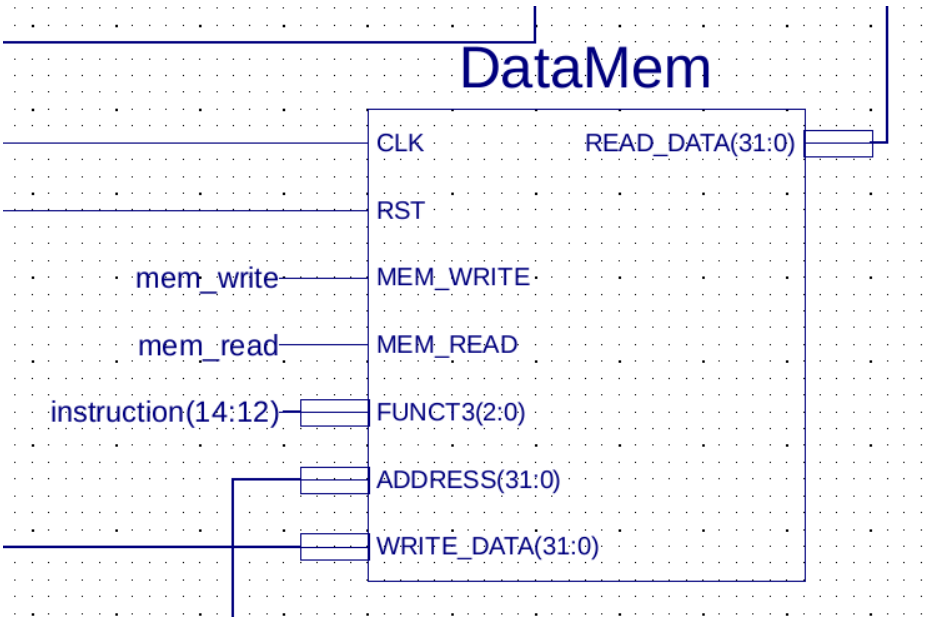
Between Register File and ALU.

Selects:

- Register operand
- Immediate value

Controlled by `alu_src`.

The ALU Source multiplexer selects the second input to the ALU. The ALU can either use data from a register or an immediate value generated by the Immediate Generator. The selection is determined by the ALUSrc control signal.



Data Memory (DataMem)

Bottom-right.

Inputs:

- Address from ALU
- Write data

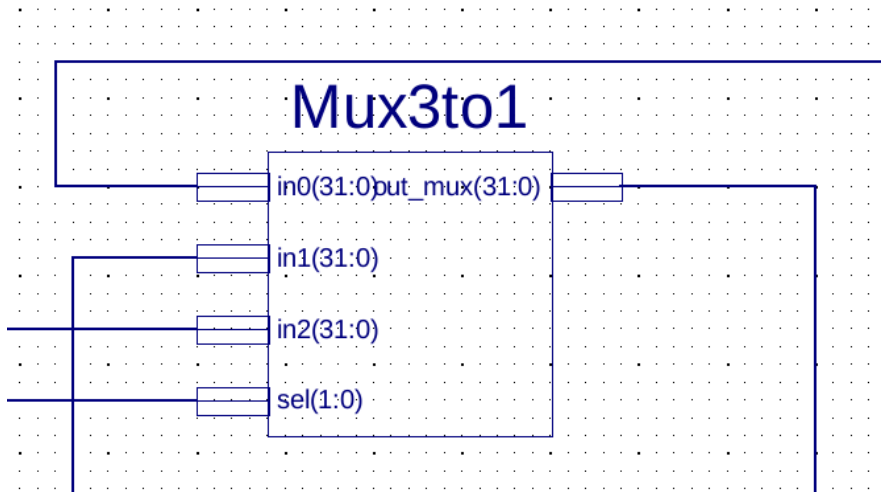
- mem_read
- mem_write

Output:

- read_data

Used for load and store instructions.

The Data Memory stores and retrieves data during load and store instructions. When memory access is required, the ALU result is used as the memory address, and data is either read from or written to memory depending on the control signals.



Mux3to1 (Write Back Selector)

Near Data Memory.

Selects data to write back to register file:

- ALU result
- Memory read data
- Possibly PC+4 (for JAL)

The Write-Back multiplexer selects the value that will be written back to the register file. The source can be the ALU result, data from memory, or PC + 4 (for jump instructions). This ensures the correct result is stored in the destination register.

Instruction Flow Observed:

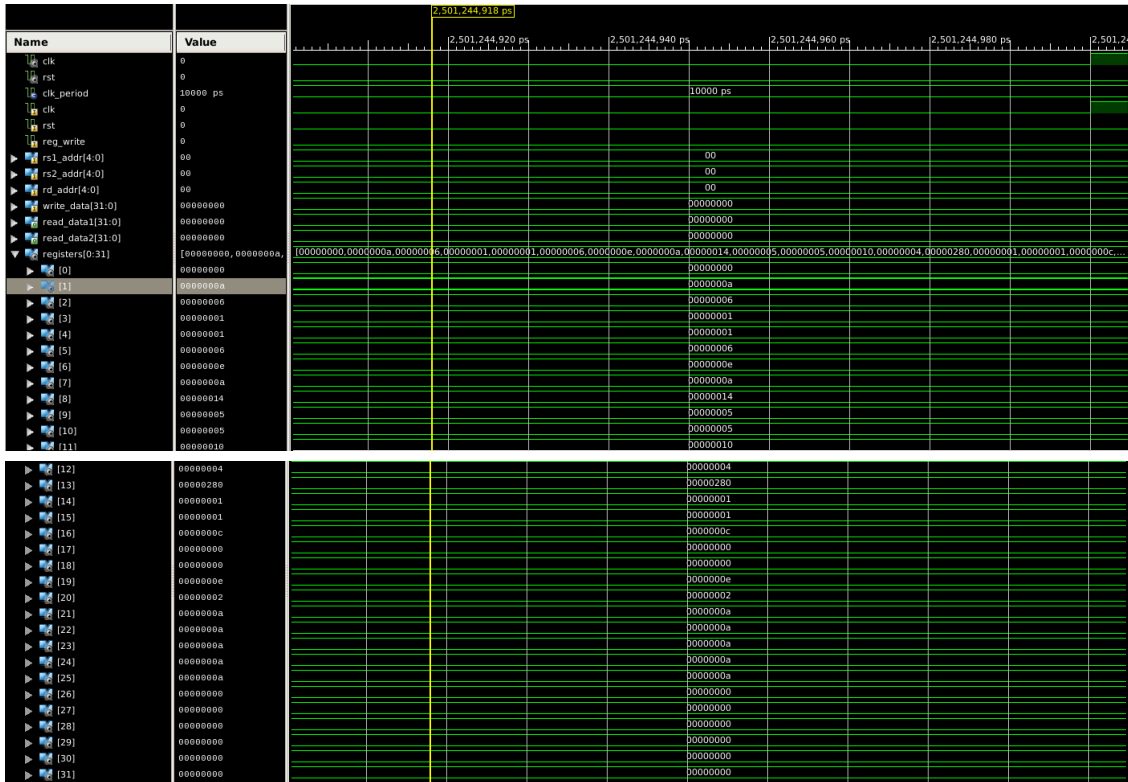
The datapath clearly implements:

1. **Fetch**
 - PC → Instruction Memory
2. **Decode**
 - Instruction → Main Control
 - Instruction → Register File
 - Instruction → ImmGen

3. Execute
- Register/Immediate → ALU
4. Memory
- ALU result → Data Memory
5. Write Back
- Selected result → Register File
6. PC Update
- Mux selects next PC (PC+4 / Branch / Jump)

The datapath shows the complete instruction execution process in a single clock cycle. Instructions are fetched from memory, decoded, executed by the ALU, optionally access data memory, and then write results back to the register file. Control signals generated by the control units coordinate the flow of data between all components.

Output Waveform



The waveform images show the simulation results of the Single-Cycle RISC-V SAP processor during execution. The waveform viewer displays the changes in signals over time, allowing verification that the processor components operate correctly and that instructions are properly executed.

Clock and Reset Signals

The clock signal (clk) drives the entire processor and synchronizes all operations. Each rising edge of the clock triggers updates in sequential components such as the Program Counter and Register File. The clock period shown in the waveform is 10,000 ps, which represents the duration of one complete clock cycle.

The reset signal (rst) is used to initialize the processor at the start of simulation. When reset is active, registers and control states are set to their initial values to ensure correct system startup before instruction execution begins.

Register Write Operation

The signal `reg_write` indicates whether data is written into the register file. When this signal is active, the value from the write-back stage is stored into the register specified by the `rd_addr` signal. The waveform confirms that register updates occur only when this control signal is enabled, ensuring proper instruction execution.

Register Address Signals

The signals `rs1_addr`, `rs2_addr`, and `rd_addr` represent the addresses of the registers being accessed in the register file.

- `rs1_addr` and `rs2_addr` specify the source registers used as operands for the ALU.
- `rd_addr` specifies the destination register where the result of the instruction will be written.

These signals demonstrate how the processor selects registers during the instruction decode stage.

Data Signals

The waveform also shows several data paths within the register file:

- `write_data[31:0]` represents the value being written back into the destination register.
- `read_data1[31:0]` and `read_data2[31:0]` represent values read from the source registers.

These signals verify that the register file correctly outputs operands for ALU operations and stores results during write-back.

Register File Contents

The waveform includes the registers array, showing the values stored in registers `x0` through `x31`. During simulation, several registers change values as instructions are executed. These updates confirm that arithmetic operations, immediate values, and memory operations are functioning correctly.

For example, some registers contain values such as `00000001`, `00000005`, `0000000A`, and `00000014`, indicating that the processor successfully performed arithmetic computations and stored the results in the appropriate registers.

Timing Behavior

The horizontal axis of the waveform represents simulation time in picoseconds (ps). As time progresses, the waveform shows how control signals and data values change with each clock cycle. Observing these transitions confirms that the processor components operate synchronously and that instruction execution follows the expected sequence.

The simulation waveform confirms the correct functionality of the SAP processor design. The signals demonstrate that the register file correctly reads and writes data, control signals activate at the appropriate times, and register values change according to the executed instructions. These results verify that the datapath and control logic operate as intended within the single-cycle architecture.

FINDINGS, OBSERVATIONS, AND COMMENTS

Through the development of the Single-Cycle RISC-V SAP Computer, the members gained a deeper understanding of how a processor truly works beyond theoretical diagrams shown in lectures. At first, concepts such as instruction

formats, control signals, and datapath flow seemed straightforward when discussed in class. However, when the implementation of each component in VHDL and connecting it together began, the members realized that even small mistakes in signal connections or bit placements could cause the entire system to malfunction. This project helped the members understand the importance of precision, especially in instruction decoding, immediate generation, and control logic design. The members also learned that the correctness of a processor depends not only on individual components working properly, but also on how well they are integrated as one system.

One of the most challenging yet meaningful experiences was debugging the datapath during simulation. There were instances where the processor did not branch correctly or where register values were not updating as expected. Through careful observation of waveform outputs and signal transitions, we were able to identify errors in control signals and ALU operations. This process strengthened the analytical and problem-solving skills of the members because they had to trace the flow of data step by step, from instruction fetch to write-back. It also helped them appreciate how control units coordinate the behavior of hardware components and how branch conditions rely on flags such as ZERO, LESS, and LESS_U. Seeing the processor successfully execute different instruction types in any order gave them a strong sense of accomplishment.

Overall, this project enhanced both the members’ technical knowledge and teamwork skills. They learned how to divide responsibilities effectively, communicate design decisions clearly, and support one another during debugging and testing. More importantly, the members developed a clearer understanding of how machine-level instructions are translated into hardware operations inside a processor. This experience allowed them to bridge the gap between theory and practical implementation, making computer architecture concepts more concrete and meaningful. The SAP computer project challenged us, but it also strengthened their confidence in designing and analyzing digital systems.

APPENDICES

Manual Calculations Per Instruction

Immediate-type instructions	
ADDI x1, x0, 10	Binary = 0000 0000 1010 0000 0000 0000 1001 0011 Hexadecimal = 0x00a00093
ADDI x2, x0, 6	Binary = 0000 0000 0110 0000 0000 0001 0001 0011 Hexadecimal = 0x00600113
SLTI x3, x1, 15	Binary = 0000 0000 1111 0000 1010 0001 1001 0011 Hexadecimal = 0x00f0a193

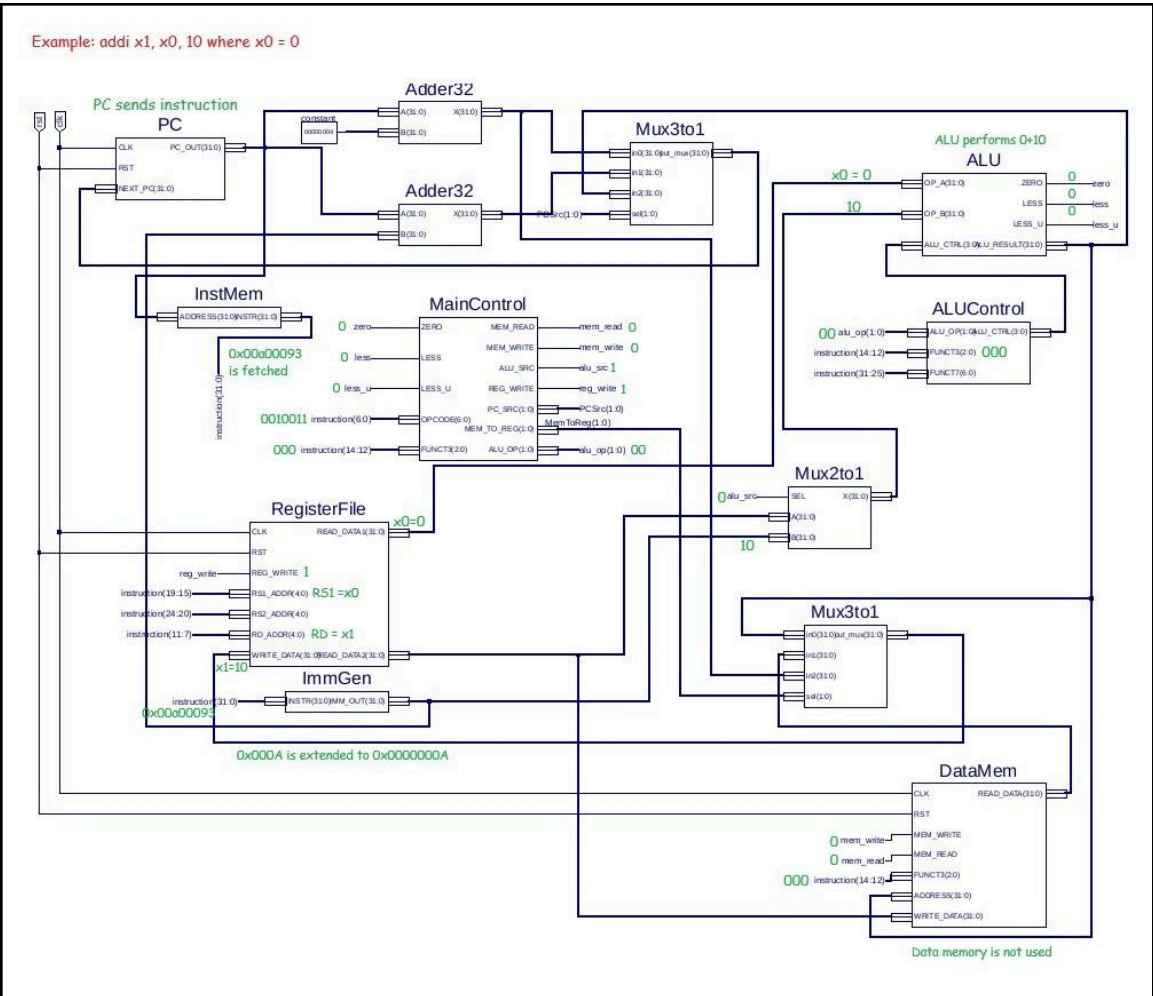
SLTIU x4, x1, 15	Binary = 0000 0000 1111 0000 1011 0010 0001 0011 Hexadecimal = 0x00f0b213
XORI x5, x1, 12	Binary = 0000 0000 1100 0000 1100 0010 1001 0011 Hexadecimal = 0x00c0c293
ORI x6, x1, 4	Binary = 0000 0000 0100 0000 1110 0011 0001 0011 Hexadecimal = 0x0040e313
ANDI x7, x1, 14	Binary = 0000 0000 1110 0000 1111 0011 1001 0011 Hexadecimal = 0x00e0f393
SLLI x8, x1, 1	Binary = 0000 0000 0001 0000 1001 0100 0001 0011 Hexadecimal = 0x00109413
SRLI x9, x1, 1	Binary = 0000 0000 0001 0000 1101 0100 1001 0011 Hexadecimal = 0x0010d493
SRAI x10, x1, 1	Binary = 0100 0000 0001 0000 1101 0101 0001 0011 Hexadecimal = 0x4010d513
Register-type instructions	
ADD x11, x1, x2	Binary = 0000 0000 0010 0000 1000 0101 1011 0011 Hexadecimal = 0x002085b3
SUB x12, x1, x2	Binary = 0100 0000 0010 0000 1000 0110 0011 0011 Hexadecimal = 0x40208633
SLL x13, x1, x2	Binary = 0000 0000 0010 0000 1001 0110 1011 0011 Hexadecimal = 0x002096b3
SLT x14, x2, x1	Binary = 0000 0000 0001 0001 0010 0111 0011 0011 Hexadecimal = 0x00112733
SLTU x15, x2, x1	Binary = 0000 0000 0001 0001 0011 0111 1011 0011 Hexadecimal = 0x001137b3
XOR x16, x1, x2	Binary = 0000 0000 0010 0000 1100 1000 0011 0011 Hexadecimal = 0x0020c833

SRL x17, x1, x2	Binary = 0000 0000 0010 0000 1101 1000 1011 0011 Hexadecimal = 0x0020d8b3
SRA x18, x1, x2	Binary = 0100 0000 0010 0000 1101 1001 0011 0011 Hexadecimal = 0x4020d933
OR x19, x1, x2	Binary = 0000 0000 0010 0000 1110 1001 1011 0011 Hexadecimal = 0x0020e9b3
AND x20, x1, x2	Binary = 0000 0000 0010 0000 1111 1010 0011 0011 Hexadecimal = 0x0020fa33
Store and Load instructions	
SB x1, 0(x0)	Binary = 0000 0000 0001 0000 0000 0000 0010 0011 Hexadecimal = 0x00100023
SH x1, 2(x0)	Binary = 0000 0000 0001 0000 0001 0001 0010 0011 Hexadecimal = 0x00101123
SW x1, 4(x0)	Binary = 0000 0000 0001 0000 0010 0010 0010 0011 Hexadecimal = 0x00102223
LB x21, 0(x0)	Binary = 0000 0000 0000 0000 0000 1010 1000 0011 Hexadecimal = 0x00000a83
LH x22, 2(x0)	Binary = 0000 0000 0010 0000 0001 1011 0000 0011 Hexadecimal = 0x00201b03
LW x23, 4(x0)	Binary = 0000 0000 0100 0000 0010 1011 1000 0011 Hexadecimal = 0x00402b83
LBU x24, 0(x0)	Binary = 0000 0000 0000 0000 0100 1100 0000 0011 Hexadecimal = 0x00004c03
LHU x25, 2(x0)	Binary = 0000 0000 0010 0000 0101 1100 1000 0011 Hexadecimal = 0x00205c83
Branch-type instruction	
BEQ x1, x1, +4	Binary = 0000 0000 0001 0000 1000 0010 0110 0011 Hexadecimal = 0x00108263

BNE x1, x2, +4	Binary = 0000 0000 0010 0000 1001 0010 0110 0011 Hexadecimal = 0x00209263
BLT x2, x1, +4	Binary = 0000 0000 0001 0001 0100 0010 0110 0011 Hexadecimal = 0x00114263
BGE x1, x2, +4	Binary = 0000 0000 0010 0000 1101 0010 0110 0011 Hexadecimal = 0x0020d263
BLTU x2, x1, +4	Binary = 0000 0000 0001 0001 0110 0010 0110 0011 Hexadecimal = 0x00116263
BGEU x1, x2, +4	Binary = 0000 0000 0010 0000 1111 0010 0110 0011 Hexadecimal = 0x0020f263
Jump instructions	
JAL x27, +4	Binary = 0000 0000 0100 0000 0000 1101 1110 1111 Hexadecimal = 0x00400def
JALR x26, 0(x0)	Binary = 0000 0000 0000 0000 0000 1101 0110 0111 Hexadecimal = 0x00000d67

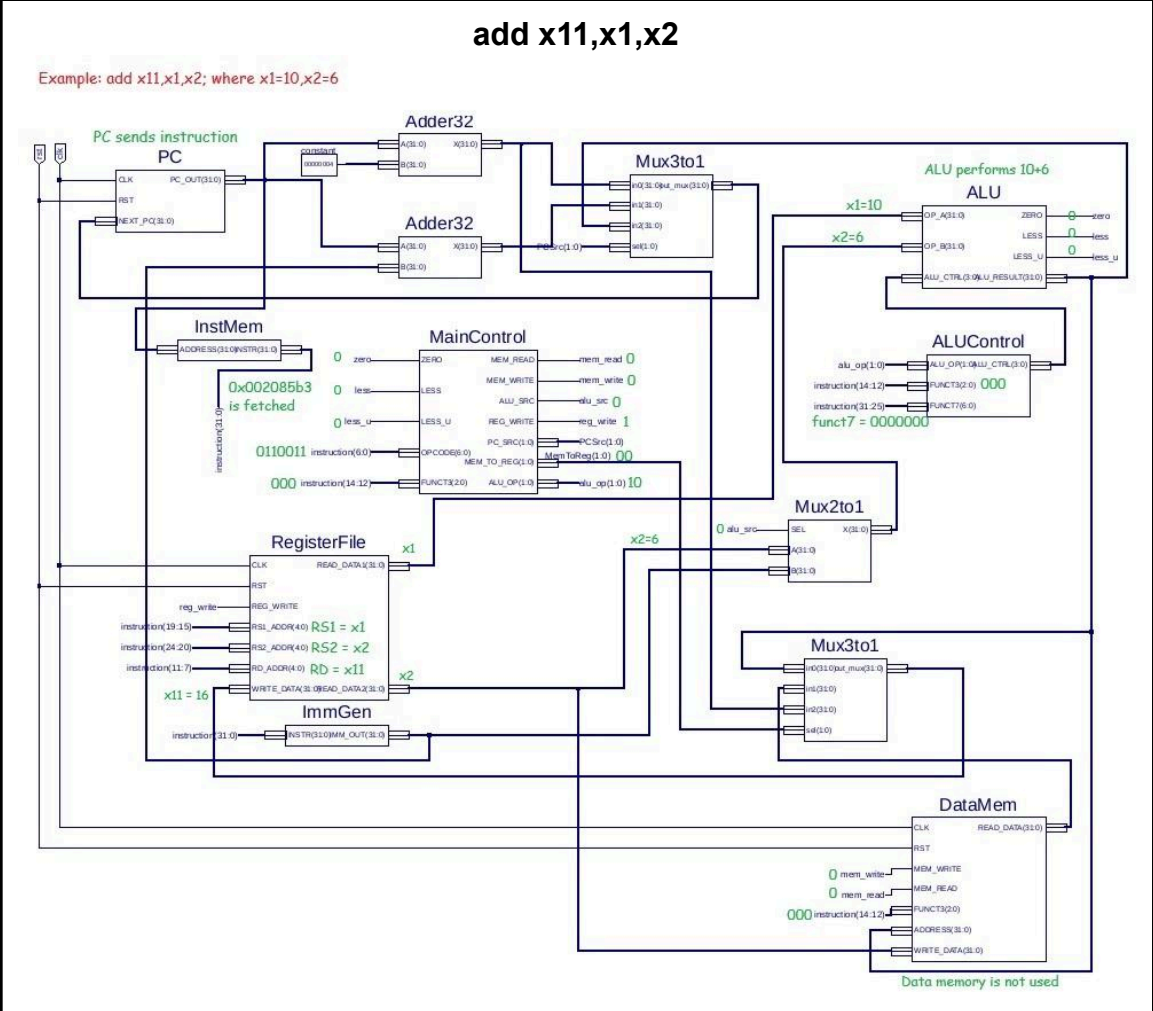
Datapath Visualization per Instruction Type

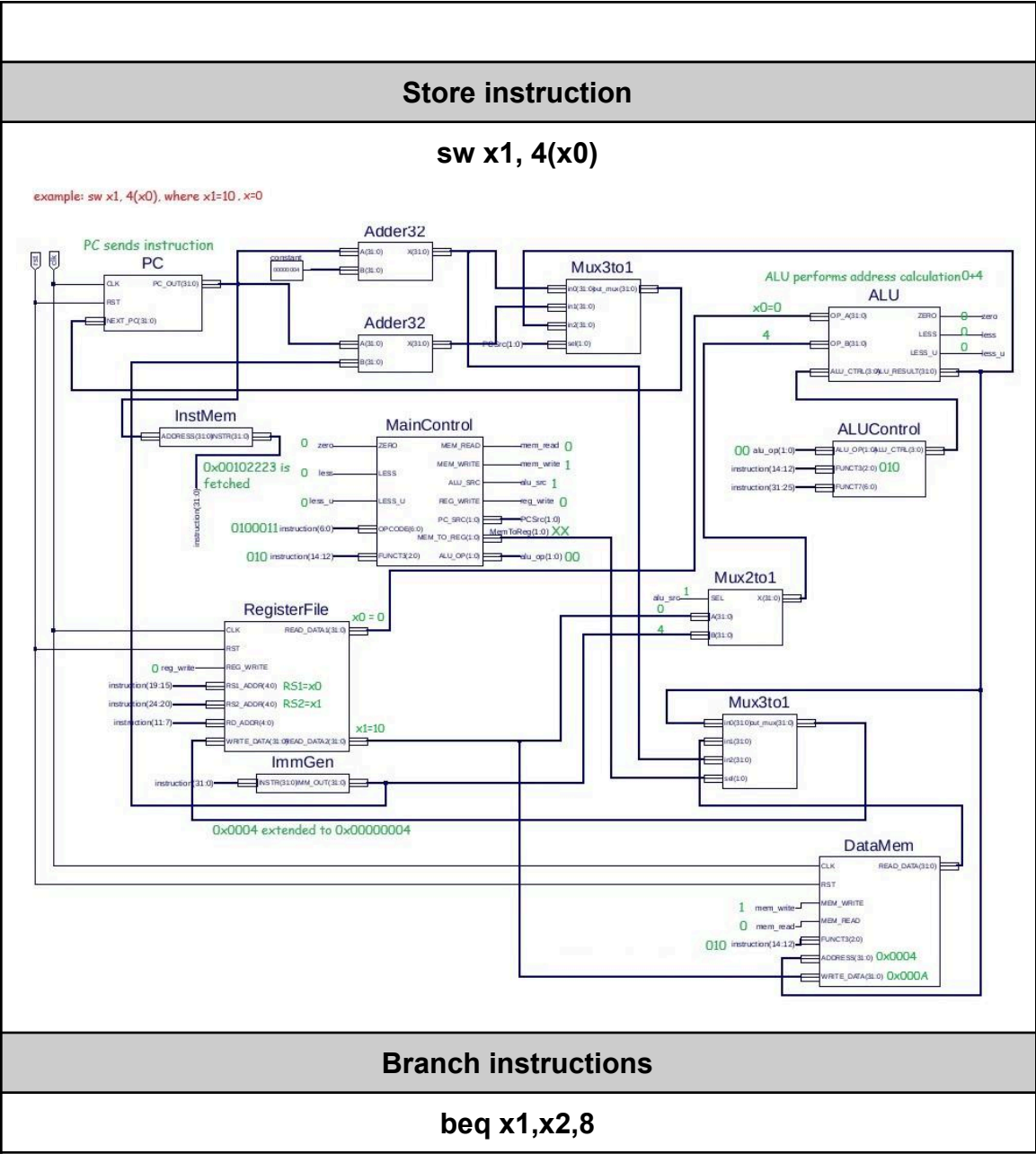
Immediate-type instructions
addi x1,x0,10



Register-type instructions

add x11,x1,x2





Branch instructions

beq x1,x2,8

jal x27,4

example: jal x27,4

The diagram illustrates the execution of the `jal x27,4` instruction. Key components and their states are as follows:

- PC (Program Counter):** Receives the instruction from the PC sends instruction block. It outputs `PC_OUT(31:0)` and `NEXT_PC(31:0)`.
- InstMem (Instruction Memory):** Receives `ADDRESS(31:0)INST(31:0)` and outputs `instruction(31:0)`. It shows the instruction `0x00400def` is fetched.
- MainControl:** Manages the instruction pipeline. It receives `instruction(31:0)` and outputs `MEM_READ`, `MEM_WRITE`, `ALU_SRC`, `ALU_OP(1:0)`, `PC_SRC(1:0)`, `MEM_TO_REG(1:0)`, `reg_write`, and `alu_op(1:0)`. It also outputs `zero`, `less`, and `less_u` signals.
- Register File:** Receives `CLK`, `READ_DATA(31:0)`, `RST`, `reg_write`, `RS1_ADDR(4:0)`, `RS2_ADDR(4:0)`, `RD_ADDR(4:0)`, and `WRITE_DATA(31:0)READ_DATA(31:0)`. It outputs `RD_ADDR(4:0)` and `RD = x27`.
- ImmGen (Immediate Generator):** Receives `instruction(31:0)` and outputs `INST(31:0)IMM_OUT(31:0)`. It shows `0x0004` is extended to `0x00000004`.
- ALU (Arithmetic Logic Unit):** Receives `OP_A(31:0)`, `OP_B(31:0)`, `ALU_CTRL(3:0)ALU_RESULT(31:0)`, and `alu_op(1:0)`. It outputs `ALU_RESULT(31:0)`. It shows `ALU is not used`.
- ALUControl:** Receives `instruction(14:12)` and `instruction(31:25)`. It outputs `ALU_OP(1:0)` and `ALU_CTRL(3:0)`.
- Mux3to1 (3-to-1 Multiplexer):** Receives `PC_OUT(31:0)`, `PC_SRC(1:0)`, `PC_OUT(31:0)`, `PC_OUT(31:0)`, and `PC_OUT(31:0)`. It outputs `PC_OUT(31:0)`.
- Mux2to1 (2-to-1 Multiplexer):** Receives `SEL`, `X(31:0)`, and `PC_OUT(31:0)`. It outputs `PC_OUT(31:0)`.
- Mux3to1 (3-to-1 Multiplexer):** Receives `PC_OUT(31:0)`, `PC_OUT(31:0)`, `PC_OUT(31:0)`, and `PC_OUT(31:0)`. It outputs `PC_OUT(31:0)`.
- DataMem (Data Memory):** Receives `CLK`, `READ_DATA(31:0)`, `RST`, `MEM_WRITE`, `MEM_READ`, `instruction(14:12)`, `instruction(31:25)`, `ADDRESS(31:0)`, and `WRITE_DATA(31:0)`. It outputs `READ_DATA(31:0)`. It shows `Data memory is not used`.

REFERENCES

Patterson, D. A., & Hennessy, J. L. (2021). *Computer Organization and Design RISC-V Edition (2nd ed.)*. Morgan Kaufmann.

RISC-V International. (2019). *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA (Version 20191213)*. RISC-V Foundation.
<https://riscv.org/technical/specifications/>

Waterman, A., & Asanović, K. (2019). *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. RISC-V Foundation.

Hennessy, J. L., & Patterson, D. A. (2019). *A New Golden Age for Computer Architecture*. *Communications of the ACM*, 62(2), 48–60.